GUÍA DE ESTUDIO · INGENIERÍA DE SOFTWARE I

Unidades I y II · 24 lecciones

Síntesis para repasar los apartados 1.1 a 2.7. Basada principalmente en las diapositivas facilitadas por el usuario. Las aclaraciones procedentes de fuentes externas se señalan como actualizaciones.

Cómo utilizarla

1. Lee una lección. 2. Cierra la guía y responde la pregunta de recuperación. 3. Abre el examen correspondiente. 4. Revisa errores y repite temas en días posteriores.

Advertencia metodológica

En las diapositivas se presentan ocho características históricas de calidad del producto. La edición ISO/IEC 25010:2023 presenta nueve. Asimismo, las láminas de Scrum mencionan duraciones concretas de sprints; la Guía Scrum 2020 indica un mes o menos.

1.1 Definiciones de ingeniería de software

Referencia: U1, pp. 3–5 · Diapositivas

La ingeniería de software no equivale únicamente a programar: integra principios, métodos, herramientas, calidad y mantenimiento para construir software que resuelva necesidades.

Bauer (1972)

Uso de principios de ingeniería bien fundados para obtener software fiable, económico y que funcione en máquinas reales.

IEEE (1993)

Aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software.

Alan Davis

Aplicación inteligente de principios, técnicas, lenguajes y herramientas para crear y mantener software que satisfaga necesidades de los usuarios a costo razonable.

Boehm (1976)

Aplicación práctica del conocimiento científico al diseño y construcción de programas y de la documentación necesaria para desarrollarlos, operarlos y mantenerlos.

Idea en común

Todas las formulaciones persiguen resultados fiables mediante un trabajo organizado; se distinguen por su énfasis en métodos, usuario, economía y documentación.

Ejemplo: Una aplicación de reservas bien ingenierizada comienza investigando necesidades, formula requisitos, implementa pruebas, se entrega y se mantiene.

Para recordar: ¿Qué palabras usa el IEEE para caracterizar su enfoque?

Error frecuente: Confundir ingeniería de software con escribir código exclusivamente.

1.2 Surgimiento, evolución y relación con otras disciplinas

Referencia: U1, pp. 6–14 · Diapositivas

La crisis del software evidenció que la complejidad creciente superaba la capacidad para planear y entregar sistemas a tiempo. La presentación sitúa la conferencia de Garmisch en 1968 como hito de la disciplina.

Primera era

Programación artesanal y poco planificada; los programas eran específicos y sus autores normalmente los mantenían.

Segunda era (mediados de 1960)

El software comenzó a distribuirse como producto y surgieron empresas dedicadas a su desarrollo.

Tercera era (mediados de 1970)

Aumentó la complejidad de los sistemas y aparecieron redes de computadoras.

Cuarta era (aprox. 1990)

El software se extendió a la economía y sociedad; las láminas destacan inteligencia artificial, redes neuronales y lógica difusa.

Disciplinas relacionadas

Matemáticas aportan pruebas y métodos formales; gestión de proyectos controla tiempo, costos y personas; ciencias de la computación enfatiza fundamentos; ingeniería de sistemas observa hardware, software y personas conjuntamente.

Áreas SWEBOK mencionadas

Requisitos, diseño, construcción, pruebas, mantenimiento, configuración, gestión, procesos, herramientas y métodos, y calidad del software.

Ejemplo: Si un equipo no estima tiempos ni gestiona cambios, corre riesgos similares a los asociados históricamente con la crisis del software.

Para recordar: ¿Qué diferencia hay entre ciencias de la computación e ingeniería del software?

Error frecuente: Pensar que 1968 es el año en que se inventó el primer programa; se trata de un hito de denominación y organización de la disciplina.

1.3 Características del software

Referencia: U1, pp. 15–18 · Diapositivas

El software es lógico, no una pieza física. Su construcción, costos y evolución no se comportan igual que los del hardware.

Se desarrolla, no se fabrica clásicamente

Se diseña y programa; reproducir una copia tiene un costo marginal muy distinto al de fabricar piezas físicas.

No se desgasta físicamente

Un programa puede fallar por defectos y cambios del entorno, pero sus instrucciones no se deterioran por uso como un mecanismo.

Gran parte se construye a medida

Las láminas hacen énfasis en soluciones construidas para necesidades concretas, en contraste con el ensamble físico.

Los costos se concentran en ingeniería

Comprensión, diseño, construcción, pruebas y mantenimiento consumen esfuerzo significativo.

Mantenimiento es indispensable

Corregir defectos y adaptarse a nuevas necesidades es trabajo técnico, aunque no exista desgaste físico.

Ejemplo: Una nueva versión del sistema operativo puede volver incompatible una aplicación intacta: el software no sufrió desgaste, pero necesita mantenimiento.

Para recordar: ¿Por qué «no se estropea» no significa «no falla»?

Error frecuente: Conclusión errónea: si el software no se desgasta, nunca requiere reparaciones.

1.4 Calidad del software según ISO/IEC 25000

Referencia: U1, pp. 19–23 · Diapositivas; ampliación ISO 25010:2023

La familia SQuaRE sirve para especificar, medir y evaluar la calidad. Para el examen basado en las láminas debes reconocer las ocho características allí expuestas.

Adecuación funcional (las láminas dicen «funcionalidad»)

Completitud, corrección y pertinencia de las funciones frente a necesidades.

Rendimiento

Tiempos de respuesta y utilización de recursos.

Compatibilidad

Coexistencia con otros productos e interoperabilidad o intercambio efectivo de información.

Fiabilidad

Madurez, disponibilidad, tolerancia a fallos y recuperación.

Usabilidad

Capacidad de aprendizaje, facilidad de uso, protección contra errores, estética y accesibilidad.

Seguridad

Confidencialidad, integridad, autenticidad, responsabilidad y no repudio, conforme al material.

Mantenibilidad

Modularidad, reutilización, análisis, modificación y prueba.

Portabilidad

Adaptabilidad e instalación/reemplazo en otros entornos.

Ejemplo: Una plataforma de reservaciones puede ser funcional pero deficiente en rendimiento si cada búsqueda tarda demasiado.

Para recordar: Clasifica estos problemas: caída frecuente, filtración de datos, interfaz difícil, lentitud.

Error frecuente: Confundir cumplimiento funcional con rapidez; son características distintas.

Actualización: Actualización externa: ISO/IEC 25010:2023 define nueve características de calidad del producto. Esa edición no debe mezclarse sin avisar con la estructura histórica de ocho características explicada en las diapositivas. Fuente: https://www.iso.org/standard/78176.html

1.5 Definición y etapas del proceso de software

Referencia: U1, pp. 24–29 · Diapositivas

Un proceso de software es un conjunto organizado de actividades y resultados para desarrollar y mantener un producto. No existe un único proceso ideal.

Especificación de requisitos

Determina funcionalidades y restricciones; suele producir un documento de requisitos. Errores iniciales se propagan.

Diseño e implementación

Define estructura, componentes, interfaces, datos y algoritmos; después se construye software ejecutable.

Validación

Comprueba que el sistema cumple especificaciones y necesidades del cliente. Incluye pruebas de componentes, sistema y aceptación.

Mantenimiento o evolución

Adecúa el software a defectos, nuevas condiciones y necesidades cambiantes.

Adaptar el proceso

Sistemas críticos suelen necesitar estructuras rigurosas; negocios cambiantes pueden beneficiarse de mayor flexibilidad.

Ejemplo: Antes de programar una reserva se especifican restricciones; luego se implementa, se verifica con pruebas y posteriormente se ajusta a nuevas políticas.

Para recordar: ¿Cuál es la diferencia entre especificar, construir, validar y evolucionar?

Error frecuente: Considerar la validación como una actividad que solo ocurre después de la entrega.

1.6 Herramientas para el desarrollo de software

Referencia: U1, pp. 30–38 · Diapositivas

Las herramientas automatizan o apoyan actividades de un proceso de software. CASE significa Ingeniería de Software Asistida por Computadora.

CASE

Incluye modeladores, diccionarios de datos, generadores de interfaces, depuradores y traductores automáticos; la integración entre herramientas permite reutilizar información.

Gestión de tareas

Jira, Trello y Asana organizan trabajo, incidencias y equipos.

Control de versiones

Git registra cambios; GitHub y GitLab hospedan repositorios y facilitan colaboración.

Integración y entrega continuas

Jenkins, Travis CI y CircleCI automatizan procesos de construcción y prueba.

Desarrollo y depuración

IDE como Eclipse, Visual Studio, PyCharm; depuradores como GDB y pdb.

Requisitos y documentación

IBM DOORS y Confluence respaldan requisitos; Swagger/OpenAPI y Sphinx documentan productos.

Pruebas

Selenium automatiza interacciones web; JUnit/TestNG permiten pruebas Java y PyTest pruebas Python.

Límites de CASE

No automatiza por completo decisiones creativas, negociación o diseño; requiere capacitación e integración.

Ejemplo: Git permite recuperar cambios, Jira mostrar tareas pendientes y PyTest ejecutar pruebas automáticas de una aplicación Python.

Para recordar: ¿Qué usarías para versiones, requisitos, pruebas web y CI/CD?

Error frecuente: Pensar que una herramienta equivale al proceso o sustituye la colaboración humana.

2.1 Definición de modelo de proceso

Referencia: U2, pp. 3–4 · Diapositivas

Un modelo de proceso es una representación abstracta y parcial de las actividades del desarrollo de software, vista desde un enfoque particular.

No es una receta universal

Puede extenderse, configurarse y adaptarse a un proyecto real.

Familias de modelos

Tradicionales, incrementales, evolutivos, proceso unificado, ágiles y formales.

Diferencia fundamental

El proceso real es el trabajo ejecutado; el modelo organiza cómo comprenderlo y gestionarlo.

Ejemplo: Un equipo puede adoptar Scrum pero complementar su trabajo con requisitos documentados y revisiones más formales.

Para recordar: ¿Por qué un modelo de proceso no describe todos los proyectos con exactitud?

Error frecuente: Confundir un modelo de desarrollo con una notación de código.

2.2 Modelos tradicionales: visión general

Referencia: U2, pp. 5–6 · Diapositivas

Las metodologías tradicionales privilegian planificación anticipada, fases definidas y documentación; en las diapositivas se describen como poco flexibles ante cambios tardíos.

Requisitos iniciales

Se busca conocer y establecer alcance al principio.

Enfoque secuencial

Una actividad principal suele cerrar antes de comenzar la siguiente.

Fortalezas

Trazabilidad documental, hitos explícitos y previsibilidad cuando los requisitos son estables.

Dificultades

Retroalimentación tardía y altos costos de cambios tardíos según contexto.

Ejemplo: Un proyecto regulado con alcance contractual estable puede valorar entregables y aprobaciones claramente documentados.

Para recordar: ¿Qué gana y qué pierde un proyecto al fijar requisitos desde el inicio?

Error frecuente: Creer que todos los enfoques tradicionales prohíben cualquier corrección; las láminas enfatizan rigidez relativa.

2.2.1 Modelo en cascada

Referencia: U2, pp. 7–9 · Diapositivas

El modelo de cascada (ciclo de vida clásico) avanza principalmente de manera secuencial, con objetivos definidos para cerrar cada fase.

1. Análisis y definición de requerimientos

Se acuerdan restricciones, funciones y metas del sistema.

2. Diseño

Se describe arquitectura, componentes, abstracciones y relaciones.

3. Implementación y pruebas unitarias

Se programa cada unidad y se verifica frente a su especificación.

4. Integración y pruebas del sistema

Se ensamblan las unidades y se valida el sistema completo para entregarlo.

5. Operación y mantenimiento

El sistema se utiliza, corrige y adapta.

Fortaleza y limitación

Fácil de comprender y planificar; poco adaptable cuando se descubren requisitos tardíamente.

Ejemplo: En un sistema de inventario con reglas muy estables, podría facilitar control y documentación de cada etapa.

Para recordar: Enumera las cinco fases del cascada en orden.

Error frecuente: Interpretar cascada como entregas funcionales cada semana; su representación original es por etapas.

2.2.2 Modelo en V

Referencia: U2, pp. 10–14 · Diapositivas

El modelo en V extiende la idea secuencial de cascada haciendo visible la correspondencia entre trabajo de desarrollo y niveles de prueba.

Lado izquierdo

Requisitos, análisis y diseño descienden hasta codificación.

Lado derecho

Las pruebas ascienden desde unidades hacia integración, sistema y aceptación.

Correspondencias

Codificación ↔ pruebas unitarias; diseño ↔ integración/sistema; requisitos ↔ aceptación (según esquema de clase).

Ventajas

Verificación planificada, revisiones por fase, documentación y responsabilidad.

Desventajas

Si los requisitos cambian mucho, replanear etapas y pruebas puede resultar costoso; el usuario observa el producto completo tarde.

Ejemplo: Si se especifica «solo usuarios activos pueden reservar», desde requisitos se prepara qué probar en aceptación.

Para recordar: ¿Qué actividad de prueba se vincula con codificación?

Error frecuente: Confundir la V con un modelo iterativo de prototipos.

2.3 Modelos incrementales: visión general

Referencia: U2, pp. 15–22 · Diapositivas

Los modelos incrementales construyen el producto por entregas funcionales sucesivas. Cada entrega añade valor utilizable y permite conocer antes el comportamiento real.

Incrementos

Cada etapa produce una versión operativa más amplia.

Priorización

Se atienden primero funciones esenciales o riesgos relevantes.

Evaluación

Los usuarios pueden probar versiones tempranas.

Riesgos

Distribuir trabajo entre incrementos y gestionar arquitectura compartida requiere experiencia.

Ejemplo: Una aplicación de hospedaje puede lanzar búsqueda primero, pagos después y notificaciones en una tercera entrega.

Para recordar: ¿En qué se distingue un incremento funcional de una fase de cascada?

Error frecuente: Creer que incremento significa solo maqueta sin funciones utilizables.

2.3.1 Modelo incremental

Referencia: U2, pp. 15–22 · Diapositivas

Combina secuencias de actividades con entregas escalonadas; el primer incremento normalmente es el núcleo funcional.

Primer incremento

Ofrece las funciones esenciales para comenzar a utilizar y evaluar el producto.

Siguientes incrementos

Amplían la solución de forma controlada hasta acercarse al conjunto de necesidades.

Beneficios

Entrega temprana, depuración en unidades de trabajo pequeñas, feedback y gestión de riesgos.

Limitaciones de las láminas

Puede ser difícil estimar el costo completo y proteger la arquitectura cuando aparecen requisitos no anticipados.

Matiz importante

Algunas diapositivas describen una arquitectura inicial completa y requisitos definidos al comienzo; esto representa el enfoque particular del material, no todos los posibles procesos incrementales.

Ejemplo: El incremento 1 permite consultar habitaciones; el 2 incorpora la solicitud de reserva; el 3 integra recordatorios.

Para recordar: ¿Qué debe entregar el incremento 1 para que aporte valor?

Error frecuente: Confundir incremento con simple repetición de trabajo sin nueva funcionalidad.

2.3.2 DRA: desarrollo rápido de aplicaciones

Referencia: U2, pp. 23–28 · Diapositivas

DRA/RAD procura periodos cortos, en las láminas se indica una referencia de 60 a 90 días, utilizando componentes, herramientas y colaboración del usuario.

Modelado de gestión

Identifica flujo de información y actores que la procesan.

Modelado de datos

Define entidades, almacenes y relaciones.

Modelado de procesos

Describe operaciones para crear, modificar, consultar y suprimir datos.

Generación de aplicaciones

Aprovecha herramientas y componentes para acelerar implementación.

Pruebas y entrega

Valida el producto y prepara su implementación.

Condiciones de éxito

Clientes disponibles, equipo comprometido, alcance manejable y herramientas adecuadas.

Ejemplo: Un formulario de trámites interno puede crearse rápidamente con herramientas visuales y componentes probados, pero aún requiere pruebas.

Para recordar: ¿Cuáles son las cinco etapas DRA presentadas?

Error frecuente: Creer que DRA significa software sin calidad ni participación del usuario.

2.4 Modelos evolutivos: visión general

Referencia: U2, pp. 29–34 · Diapositivas

En el enfoque evolutivo, una versión inicial se refina a partir de comentarios del usuario. Especificación, desarrollo y validación se entrelazan.

Desarrollo exploratorio

Empieza con lo bien comprendido y evoluciona hacia un producto final con el cliente.

Prototipos desechables

Experimentan para comprender requisitos inciertos; el prototipo no tiene por qué convertirse en el producto.

Ventaja

Hace posible incorporar descubrimientos durante el proyecto.

Riesgos

El avance puede ser difícil de medir y los cambios continuos pueden degradar la estructura si no se controla.

Ejemplo: Un usuario prueba una pantalla de reserva, descubre que necesita filtros y el equipo ajusta una nueva versión.

Para recordar: ¿Qué diferencia al desarrollo exploratorio de un prototipo desechable?

Error frecuente: Confundir evolución del producto con ausencia total de arquitectura.

2.4.1 Modelo de prototipos

Referencia: U2, pp. 35–39 · Diapositivas

Un prototipo es una representación preliminar que permite aclarar expectativas, descubrir necesidades y evaluar ideas con el cliente.

Recopilación

Identificar objetivos y requisitos conocidos.

Construcción rápida

Crear una maqueta o sistema parcial centrado en los aspectos visibles o inciertos.

Evaluación

Mostrarla al cliente y recibir observaciones.

Refinamiento

Corregir la comprensión y repetir el ciclo si es necesario.

Precaución

Un prototipo pensado solo para aprender no debería ponerse en producción sin ingeniería y pruebas suficientes.

Ejemplo: Antes de construir el módulo completo de reservaciones, se prueba un prototipo de calendario de disponibilidad.

Para recordar: ¿Por qué no siempre conviene convertir directamente el prototipo en producto final?

Error frecuente: Asumir que cualquier prototipo necesariamente debe conservarse en la arquitectura final.

2.4.2 Modelo en espiral

Referencia: U2, pp. 40–44 · Diapositivas

Propuesto por Barry Boehm, organiza ciclos sucesivos y coloca el análisis del riesgo en el centro de las decisiones de desarrollo.

1. Definición de objetivos

Establecer alternativas, metas y restricciones del ciclo.

2. Evaluación y reducción de riesgos

Analizar riesgos de las alternativas; usar prototipos o experimentos cuando sea útil.

3. Desarrollo y validación

Construir y comprobar la solución seleccionada.

4. Planificación del siguiente ciclo

Revisar lo obtenido y determinar actividades de la próxima vuelta.

Fortaleza

Adapta técnica y esfuerzo al riesgo del proyecto.

Limitación

Exige capacidad para identificar riesgos y gestionar el proceso.

Ejemplo: Si un componente externo podría fallar, el equipo primero realiza un prototipo técnico y prueba ese riesgo antes de construir todo el sistema.

Para recordar: ¿Qué se hace en el segundo sector de la espiral y por qué?

Error frecuente: Confundir espiral con cascada repetida sin evaluación de riesgos.

2.4.3 Desarrollo concurrente

Referencia: U2, pp. 45–49 · Diapositivas

El desarrollo concurrente representa varias actividades en curso simultáneamente, cada una en un estado propio.

Actividades coexistentes

Comunicación, modelado, construcción y otras tareas pueden avanzar en paralelo.

Estados

Una actividad puede estar en desarrollo, en revisión o a la espera de cambios según la representación.

Beneficio

Permite visibilidad del estado real y cooperación multidisciplinaria.

Reto

Exige sincronizar información y manejar dependencias entre equipos.

Ejemplo: Mientras un grupo valida una interfaz, otro implementa servicios; ambos coordinan cambios mediante acuerdos y pruebas.

Para recordar: ¿Qué significa que dos actividades existan de manera concurrente?

Error frecuente: Confundir concurrencia de tareas con ausencia de coordinación.

2.5 Proceso unificado (RUP)

Referencia: U2, pp. 50–64 · Diapositivas

El Proceso Unificado, en las diapositivas presentado mediante RUP, utiliza casos de uso, arquitectura y desarrollo iterativo e incremental; emplea UML para representar vistas del sistema.

Tres rasgos

Dirigido por casos de uso, centrado en arquitectura, iterativo e incremental.

Inicio

Justificación del negocio, actores, alcance inicial y viabilidad.

Elaboración

Comprensión del dominio, arquitectura, planificación y riesgos principales.

Construcción

Desarrollo de componentes, integración, implementación y pruebas.

Transición

Paso del producto al entorno de usuarios, operación y adopción.

Mejores prácticas

Desarrollo iterativo, gestión de requisitos, arquitectura por componentes, modelado visual, verificación de calidad y control de cambios.

UML

Lenguaje de modelado empleado para documentar y visualizar distintas perspectivas del sistema.

Ejemplo: En un sistema de biblioteca, los casos de uso de préstamo guían requisitos y la arquitectura de servicios se estabiliza en elaboración.

Para recordar: ¿Cuáles son las cuatro fases de RUP y sus tres rasgos?

Error frecuente: Confundir las cuatro fases RUP con actividades exclusivas: en cada fase pueden existir diversos flujos de trabajo.

2.6 Modelos ágiles: visión general

Referencia: U2, pp. 65–72 · Diapositivas

Las metodologías ágiles se apoyan en el Manifiesto Ágil (2001). Su propósito es entregar valor y responder a cambios con colaboración y aprendizaje frecuente.

Valores del manifiesto

Individuos e interacciones; software funcionando; colaboración con el cliente; respuesta al cambio, valorados más que sus contrapartes tradicionales.

No excluyen disciplina

Procesos, herramientas, documentación y contratos siguen siendo útiles, pero no desplazan el valor del producto y la colaboración.

Rasgos de las láminas

Iteraciones, mejora continua, pruebas, priorización por valor y colaboración frecuente.

Diversidad

Scrum, XP, DSDM y otros enfoques tienen estructuras y prácticas diferentes.

Ejemplo: Un cliente aprende tras usar una versión y modifica prioridades; el equipo ajusta su siguiente entrega sin perder control.

Para recordar: ¿Qué se valora más: responder al cambio o seguir ciegamente el plan?

Error frecuente: Pensar que «ágil» significa desarrollar sin documentación o sin pruebas.

2.6.1 Principios de los modelos ágiles

Referencia: U2, pp. 67–71 · Diapositivas

Los doce principios del Manifiesto explican cómo convertir los cuatro valores en decisiones cotidianas de desarrollo.

Satisfacción y valor

Entregar software útil temprano y continuamente.

Cambio

Aceptar cambios incluso cuando el proyecto ya avanza.

Frecuencia

Entregar productos que funcionan con periodos cortos.

Colaboración

Negocio y desarrollo deben trabajar juntos.

Personas

Confiar en equipos motivados y facilitar conversaciones efectivas.

Progreso

Medirlo fundamentalmente por software funcionando.

Sostenibilidad

Mantener un ritmo razonable sin desgaste del equipo.

Excelencia

Cuidar diseño y calidad técnica.

Simplicidad

Evitar trabajo innecesario.

Autonomía y mejora

Equipos autoorganizados y reflexiones periódicas para mejorar.

Ejemplo: Una retrospectiva identifica que se retrasan pruebas; el equipo acuerda automatizarlas en la siguiente iteración.

Para recordar: ¿Por qué el manifiesto habla de software funcionando como medida de progreso?

Error frecuente: Creer que «aceptar cambios» significa no definir una meta o no priorizar.

2.6.2 Programación extrema (XP)

Referencia: U2, pp. 73–87 · Diapositivas

XP, asociada a Kent Beck, enfatiza comunicación, simplicidad, retroalimentación, coraje y prácticas técnicas que sostienen cambios frecuentes.

Historias de usuario

El cliente expresa funcionalidades y prioridades con descripciones breves.

Roles

Programadores construyen y prueban; cliente prioriza y valida; coach ayuda a mantener las prácticas.

Ciclo explicado

Exploración, planificación de entregas, iteraciones, producción, mantenimiento y finalización del proyecto.

Programación en parejas

Dos personas colaboran en el mismo trabajo de código.

Pruebas e integración continua

Automatizar verificación y fusionar código regularmente reduce defectos difíciles de aislar.

Refactorización y diseño simple

Mejorar estructura sin alterar comportamiento, evitar complejidad innecesaria.

Propiedad colectiva, estándares y ritmo sostenible

Fomentan trabajo de equipo, legibilidad y continuidad.

Ejemplo: Para una historia «como alumno quiero consultar mis calificaciones», el cliente define aceptación y el equipo desarrolla con pruebas automatizadas.

Para recordar: ¿Qué relación hay entre pruebas automatizadas y cambios frecuentes en XP?

Error frecuente: Confundir XP con solo programar rápido sin disciplina de pruebas.

2.6.3 DSDM: método de desarrollo de sistemas dinámicos

Referencia: U2, pp. 88–98 · Diapositivas

DSDM surgió en 1994 como evolución de RAD; combina desarrollo iterativo e incremental, tiempos delimitados y participación activa del negocio.

Ocho principios

Enfocarse en el negocio; entregar a tiempo; colaborar; no comprometer la calidad; construir gradualmente desde bases firmes; desarrollar iterativamente; comunicar de forma continua y clara; demostrar control.

Timeboxing

Trabajar con periodos de duración fija para tomar decisiones realistas sobre alcance y prioridades.

Fases históricas en diapositivas

Se describen estudio de viabilidad, estudio del negocio, modelado funcional, diseño/construcción e implementación.

Vista ampliada del propio material

Más adelante se presenta otra organización del ciclo con preproyecto, factibilidad, fundamentos, exploración, ingeniería, despliegue y postproyecto: corresponde a una exposición de versiones/variantes de DSDM.

Necesidad clave

Participación de usuarios y claridad del valor esperado.

Ejemplo: En una entrega con fecha fija, el equipo prioriza lo esencial, sin reducir el estándar de calidad definido.

Para recordar: Enumera cuatro principios DSDM y explica timeboxing.

Error frecuente: Interpretar entrega a tiempo como permiso para sacrificar calidad.

2.6.4 Scrum

Referencia: U2, pp. 99–108 · Diapositivas; Guía Scrum 2020

Scrum organiza el trabajo complejo con un equipo responsable, sprints, inspección y adaptación. Las láminas cubren roles, artefactos, eventos y valores.

Responsabilidades

Product Owner optimiza valor y ordena trabajo; Scrum Master ayuda a comprender Scrum y eliminar impedimentos; Developers crean incrementos.

Artefactos

Product Backlog: trabajo del producto; Sprint Backlog: plan de trabajo del sprint; Incremento: resultado utilizable.

Eventos

Sprint Planning, Daily Scrum, Sprint Review y Sprint Retrospective ocurren dentro del Sprint.

Revisión frente a retrospectiva

Review inspecciona el resultado y adapta el producto; retrospectiva identifica mejoras del modo de trabajo.

Valores

Compromiso, enfoque, apertura, respeto y valentía.

Duración

Las láminas incluyen referencias a sprints de 30 días y también de unas dos semanas; la Guía Scrum 2020 vigente especifica uno mes o menos, no una duración única.

Reunión diaria

Evento breve, orientado a inspeccionar progreso hacia el objetivo del sprint y adaptar el plan.

Ejemplo: El Product Backlog tiene 20 mejoras; en Sprint Planning se seleccionan 4; el equipo entrega una versión usable y la muestra en la Review.

Para recordar: ¿Qué diferencia hay entre Product Backlog, Sprint Backlog e incremento?

Error frecuente: Confundir la reunión diaria con un informe formal obligatorio al jefe.

Actualización: Ampliación externa para versiones y duración: https://scrumguides.org/download.html

2.7 Modelos formales

Referencia: U2, pp. 109–119 · Diapositivas

Los métodos formales emplean lenguajes y argumentos matemáticamente rigurosos para especificar, diseñar y verificar propiedades de sistemas.

Fases de las láminas

Análisis de requisitos; especificación formal; transformación; integración del sistema final.

Especificaciones lógicas

Lógica de primer orden y teoría de conjuntos expresan estados, operaciones y propiedades.

Especificaciones algebraicas

Definen tipos y operaciones mediante axiomas o ecuaciones.

Álgebra de procesos

Permite describir interacción y coordinación entre procesos concurrentes.

Redes de Petri

Modelan concurrencia con lugares, marcas y transiciones.

Lógica temporal

Formula propiedades de sistemas reactivos y eventos en el tiempo.

Aplicación

Especialmente útil cuando las fallas pueden comprometer seguridad o fiabilidad de sistemas críticos.

Costo

Requiere conocimiento especializado y puede demandar inversión elevada.

Ejemplo: En señalización ferroviaria se modela formalmente que no pueden autorizarse dos trenes a ocupar el mismo segmento al mismo tiempo.

Para recordar: ¿Qué diferencia hay entre una red de Petri y una especificación algebraica?

Error frecuente: Considerar los métodos formales simples diagramas sin semántica matemática.

Referencias y lectura complementaria

Fuentes de base: presentación Unidad I, presentación Unidad II y temario oficial de Ingeniería de Software I, Universidad de la Sierra Juárez.

• Whitten, Jeffrey L.; Bentley, Lonnie D. Análisis de sistemas: diseño y métodos. McGraw-Hill, 2008, 7.ª ed.

• Kendall, Kenneth E.; Kendall, Julie E. Análisis y diseño de sistemas. Pearson Educación, 2011, 8.ª ed.

• Schach, Stephen R. Análisis y diseño orientado a objetos: con UML y el proceso unificado. McGraw-Hill, 2005.

• Booch, Grady; Rumbaugh, James; Jacobson, Ivar. El lenguaje unificado de modelado: Guía del usuario. Addison Wesley, 2006, 2.ª ed.

• Weitzenfeld, Alfredo. Ingeniería de software orientada a objetos con UML, Java e Internet. Thomson, 2005.

• Bruegge, Bernd; Dutoit, Allen H. Ingeniería de software orientado a objetos. Pearson Educación, 2002.

• Sommerville, Ian. Ingeniería de software. Pearson Educación, 2011, 9.ª ed.

• Braude, Eric J. Ingeniería de software: una perspectiva orientada a objetos. Alfaomega, 2003.

• Pfleeger, Shari L. Ingeniería del software: teoría y práctica. Prentice Hall, 2002.

ISO/IEC 25010:2023 – Modelo de calidad del producto — https://www.iso.org/standard/78176.html

Guía Scrum oficial 2020 — https://scrumguides.org/download.html

Nielsen Norman Group — 10 heurísticas de usabilidad — https://www.nngroup.com/articles/ten-usability-heuristics/

Dunlosky et al., 2013. Técnicas eficaces de estudio. DOI 10.1177/1529100612453266 — https://doi.org/10.1177/1529100612453266

Roediger y Karpicke, 2006. Test-enhanced learning. DOI 10.1111/j.1467-9280.2006.01693.x — https://doi.org/10.1111/j.1467-9280.2006.01693.x